Four further proof simplification ideas

for `DiophantineConsistency`

Review, mathematical arguments, and checked Lean cores

26 October 2026 Prepared for Vladimir Reshetnikov

Recommendation. Start with the local shifted-numerator replacement (C), then remove the redundant primitive-recursiveness certification route (A). Develop the boundary-carry integration (B) as the largest conceptual simplification. Treat the classical constant-predicate shortcut (D) as optional: it simplifies a proposition-valued endpoint while changing that proof's computational provenance.

These are four narrowly stated proposals that survived a comparison against the requested prior-work corpus. Earlier abstractions are treated as available; new names, tactic changes, generic telescoping, common residue certificates, and previously advertised future directions receive no new credit.

	Proposal	Residual work removed	Evidence and size
A	Consume the supplied scalar realizer	A second, separately maintained proof that the same test is primitive recursive	Checked bridge; candidate migration removes 213 snapshot lines
B	Use equal boundary carries	The coarse numerical nonwrapping proof needed before transporting a dummy increment	Four checked core lemmas; production adapter proposed
C	Bound the shifted numerator jointly	A chain of separate numerator and denominator estimates	Complete exact-signature proof checked; 10 lines removed
D	Constant-predicate recursive enumerability	Search construction inside an existence-only theorem	Checked generic core; optional original-source patch removes 10 lines including documentation

Validation boundary. Four principal Lean files and a small post-migration template compiled successfully with Lean 4.32.0 and the repository's pinned dependencies. Their axiom reports contain no `sorryAx`. The full production import closure and the candidate migration have *not* been rebuilt. Static patch applicability, unchanged executable source text, an acyclic import overlay, and exact source counts are separate evidence; they are not substitutes for a production build.

Counting boundary. The 213-line figure compares two original files with all three files in the proposed migration. It includes the new module and helper/documentation costs. It is not an estimate of the incremental saving after every earlier proposal is applied. The optional D patch overlaps A and must not be added to A's count. No total saving or compilation speedup is claimed.

Source and comparison cutoff. Production inspection began at `9d9d1123b39927d30363f1dcd53c23f0575c30ee`. The final comparison uses `60fcb2a69b7f1fdd4c3592ea5f164223cad50abe`. All eight relevant target/boundary files have identical Git blobs at the two pins. The final corpus contains 43 placed packets, a–zq; `docs/incoming` contains only its README. This is a bounded overlap finding at that commit, not a claim of mathematical priority.

Contents

1 Scope and the non-repetition rule **3**

2 A: derive computability from the supplied program **4**

2.1 The duplicated obligation . . . 4

2.2 Break the import cycle at the semantic layer . . . 4

2.3 Why this remains useful after earlier simplifications . . . 5

2.4 Adoption gate . . . 5

3 B: replace numerical safety by exact boundary conservation **5**

3.1 The proof obligation that can disappear . . . 5

3.2 The exact conservation equation . . . 6

3.3 Only the perturbation needs to stay inside the period . . . 6

3.4 Concrete integration order . . . 7

3.5 Limits and payoff . . . 7

4 C: one joint envelope for the shifted numerator **8**

4.1 Exact target and unchanged statement . . . 8

4.2 Derivation . . . 8

4.3 Measured improvement and validation . . . 9

5 D: optional classical proof for a constant predicate **9**

5.1 Why the stated endpoint admits a much shorter proof . . . 9

5.2 The non-effective boundary is substantive . . . 10

5.3 Compatibility with the choice-defined polynomial branch . . . 10

6 Evidence, reproduction, and adoption priorities **11**

6.1 What was checked . . . 11

6.2 Reproduce and integrate . . . 11

6.3 Open implementation questions . . . 12

Source map and archive contents **12**

1 Scope and the non-repetition rule

The source survey covered 520 maintained Lean modules, with 82,035 physical lines in the inspected tree. The comparison initially covered 36 placed packets and five incoming ZIP archives. During the review, those five archives were placed as packets zk–zo, and two new packets, zp and zq, arrived. All 328 retained nonadministrative payload files in zk–zo were matched byte for byte against members of the originally inspected archives. The final refresh examined 37 new text artifacts in zp and zq, including supporting notes, patches, prototypes, and further questions. The archive records the identities and the review decisions.

The audit used report outlines and inherited exclusion ledgers to map earlier mechanisms, then read candidate-relevant proposal bodies and supporting material. Searches included plain-text notes, not just the main reports. An earlier optional or deferred idea counts as prior work. An unchanged source snapshot included in a packet does not, by itself, constitute a proposed simplification. The large incoming README's historical external links were not recursively retrieved. These boundaries matter: the audit is extensive but finite, and cannot establish absolute novelty.

"Assume already applied" is interpreted as an assumption that each earlier advertised interface or mathematical mechanism is available where consistent. Some earlier packets offer incompatible alternatives, so no fictitious fully merged tree is used to measure savings. The question for each retained proposal is: *what distinct obligation still disappears when the earlier interfaces are available?*

	Closest earlier mechanism, assumed available	Difference retained in this report
A	Scoped `fun_prop` for extensional primitive-recursion proofs; shared actual scalar builders	Delete the separate extensional certification route by consuming the already supplied data-valued realizer
B	Exact carry telescoping, reset scans, and finite update linearity	Use equal actual boundary carries and exact perturbation support to remove the quantitative safety prerequisite
C	General coefficient envelopes and positive-margin arguments	A complete local replacement for the particular joint numerator in `shifted_Q_bounds`
D	Inventories of the recursive-enumerability endpoint	A classical constant-predicate proof, with an explicit change in provenance and no effective construction claim

For A, see packet i's checker notes and packet e's shared-program discussion. For B, exact telescoping in packet j, the local carry construction in packet zo, section C, and finite update transport in packet zn, section C are predecessors, not new proposals here. Packet zn retains the quantitative nonwrapping hypothesis. The later zp and zq reports also retain or reuse physical nonwrapping arguments rather than replacing them with boundary conservation.

Several tempting candidates were discarded. Generic ordered-semiring division uniqueness was already raised in a supporting `.txt` note, despite not being a headline recommendation. Explicit zero-seeded per-cell carries, a common-modulus kernel certificate, multiple-dummy batching as such, and direct modulo-16 bit simplification were also already proposed. Successful Lean experiments for an excluded idea do not restore novelty. The detailed ledger is in `audit/novelty-audit.md` and its final-refresh addendum.

2 A: derive computability from the supplied program

2.1 The duplicated obligation

Computability.lean currently certifies primitive recursiveness through a long private chain over encoded enumerator states and bounded execution. Separately, DirectScalarEnumerator.lean constructs an actual scalar primitive-recursive program realizing the same executable test. The second certificate is stronger: it supplies program data and a correctness equation, rather than only a proposition that some code exists.

The relevant existing interface in ScalarPRCode.lean is, schematically,

$$\mathrm{Realizes}(f) = \{c : \mathrm{ScalarPRCode} \mid \forall n,\ c.\,\mathrm{eval}(n) = f(n)\}.$$

Every scalar code already has a theorem `c.primrec` certifying its evaluation function. Consequently the extra primitive-recursiveness certificate follows directly:

```
theorem realizes_natPrimrec {f : Nat -> Nat}
    (p : ScalarPRCode.Realizes f) : Nat.Primrec f :=
  p.val.primrec.of_eq p.property
```

The supplied core checks this against the actual repository definition of `ScalarPRCode`, freshly compiled from the inspected source. It does not replace that definition with a simplified model. The printed axiom dependency of this bridge is only `Quot.sound`.

The exact production endpoint becomes:

```
theorem haltingTest_natPrimrec : Nat.Primrec haltingTest :=
  DirectEnumerator.program.val.primrec.of_eq
    DirectEnumerator.program.property
```

This uses `DirectEnumerator.program` as supplied. It does not extract executable data from a proposition-valued `Primrec` proof, invoke classical choice to synthesize a program, or replace the literal enumerator.

2.2 Break the import cycle at the semantic layer

A direct substitution in the current file would be cyclic: the program construction imports the file that presently contains both the test's semantics and its computability proof. The candidate migration separates those responsibilities with one lower module.

Module	Role and dependency
`EnumeratorSemantics`	Original bounded run, initial state, executable test, and halting/refutation equivalence; depends on the enumerator semantics
`DirectScalarEnumerator`	Original program construction; imports `EnumeratorSemantics`
`Computability`	Public primitive-recursiveness and recursive-enumerability results; imports `DirectScalarEnumerator`

The migration retains the original small state equivalence and private `Primcodable`/`DecidableEq` scaffold. That conservatively preserves the encoding and decision data used by the copied definitions. The text of `DirectScalarEnumerator` after its imports is unchanged. All executable definitions in the extracted semantic layer are copied from the original source; the change is the route by which their computability is certified.

There is no need to abandon the actual-search proof of recursive enumerability. Once the scalar test is primitive recursive, its Boolean equality-to-one test is primitive recursive. Apply `Partrec.rfind` to search for a successful time and use the existing equivalence

$$(\exists t,\ \mathrm{haltingTest}(t) = 1) \quad \longleftrightarrow \quad C222\ F.$$

The checked core includes this generic search argument and its composition with a supplied realizer. The proposed production file follows the same route. Thus A can preserve the documented provenance of the r.e. theorem, independently of optional proposal D.

2.3 Why this remains useful after earlier simplifications

Earlier `fun_prop` automation can make each extensional proof shorter. Shared program builders can make the actual scalar implementation cleaner. With both assumed applied, a second proof chain over the enumerator is still a second chain to maintain. A removes that obligation entirely. The import split is a prerequisite, not a separate idea or a separate savings claim.

The candidate removes 32 private `pr_*` theorem declarations and the separate `rawStep` proof route in the original computability file. It preserves the three public certification endpoints and re-exports the moved public definitions through imports. A search of current consumers found the public endpoints used from `Bridge.lean` and `checks/ZFCAxioms.lean`; no client of a removed private declaration was found. The 520-module local import overlay is acyclic. The certification endpoints remain exported through `Computability`; importing `DirectScalarEnumerator` alone no longer exposes those downstream theorems, and no current affected consumer was found.

Accounting unit	Original	Candidate
All affected production files	2	3
Physical source lines, including new module and documentation	608	395
Net physical lines removed		213

These counts describe the included snapshot patch. They are not an additive prediction after earlier automation has shortened those same declarations. The robust post-prior-work benefit is one certification route instead of two.

2.4 Adoption gate

The archive contains a full candidate diff and all three resulting files. Static applicability and the acyclic overlay were checked; the production migration was not elaborated as a whole. Before adoption, build the three affected modules and the ordinary project checks, verify public theorem signatures, and re-run any existing executable-code or witness-count `rfl` checks. Private names change when moved between modules, so source preservation alone is not a blanket claim about every future definitional comparison. Retaining the original state instances is a deliberate compatibility measure. No compile-time improvement has been measured.

3 B: replace numerical safety by exact boundary conservation

3.1 The proof obligation that can disappear

CurrentDummyMutation2011.normalized_increment presently upgrades a modular statement about a dummy mutation to an ordinary integer equality using a numerical hypothesis of the form

$$F + (K + 2^W)\delta < \rho^{2011N} - 1.$$

CurrentDummySafety2011 establishes this through kernel-magnitude, increment, quotient, and short-word bounds. The proposed replacement establishes the ordinary equality directly from two exact facts: the perturbation itself does not cross the cyclic boundary, and the old and new normalization have the same boundary carry.

This does not remove the actual Boolean and physical-placement hypotheses. It removes the need to prove a large quantitative bound *before* transferring the mutation to normalized words. For an initially unused dummy, nonwrapping follows afterward from the existing strict bound on the new normalized word. The stronger exported theorem without that premise requires separate treatment, explained below.

3.2 The exact conservation equation

Take a length-m normalization with raw entries a_i, normalized digits d_i, and carry sequence $q_0, \dots, q_m$ satisfying

$$a_i + q_i = d_i + \rho q_{i+1} \qquad (0 \le i < m).$$

Multiplying by ρ^i and summing cancels the interior carries:

$$\sum_{i<m} a_i \rho^i + q_0 = \sum_{i<m} d_i \rho^i + q_m \rho^m. \tag{1}$$

For two normalizations, equal incoming carries and equal outgoing carries make the raw-word increment equal to the normalized-word increment. In a cyclic normalization, the two endpoints of each sequence are the same boundary carry b; equation (1) becomes

$$\mathrm{rawWord} = \mathrm{digitWord} + b(\rho^m - 1).$$

Thus equality of the old and new b is sufficient. There is no coefficient-mass estimate in this reasoning.

For the actual assembled Boolean words, CurrentWordTopBounds2011.top_carry_digits proves the outgoing carry at every top field is zero. Its `last_top_val` lemma identifies the canonical last field. Both the old word and the updated Boolean word therefore have $b = 0$. This fact is independent of the large safety inequality being replaced.

The finite telescope is already part of the assumed prior machinery. The supplied `lean/BoundaryConservation.lean` verifies a compact version to make the mathematical interface explicit; it is not being credited as a new telescope API.

3.3 Only the perturbation needs to stay inside the period

Use the actual constants and positions:

$$\rho = 2^{47}, \quad m = 2011N, \quad W = 47 \cdot 2011s, \quad p = 977 + 2011j, \quad \delta = \rho^p.$$

The current hypotheses include $j + 1 < N$ and $j + s < N$. Every literal kernel exponent e is at most 2020. Therefore

$$p + e \le 977 + 2011j + 2020 < 2011N,$$
$$p + 2011s = 977 + 2011(j + s) < 2011N.$$

The first strict inequality uses the extra full cell guaranteed by $j+1 < N$; $977+2020 = 2997 < 2 \cdot 2011$. The second uses $977 < 2011$. These are bounds on physical support, not on the numerical size of K or of the whole assembled word.

Adding one initially-zero dummy changes the raw word by a translated copy of the literal kernel plus the whole-field temporal monomial. Convolution is additive. For the rotation term, apply `rotation_whole` separately to the old and new Boolean assemblies; each becomes an ordinary shift by $2011s$. This establishes the needed difference identity without asserting that binary rotation is additive on arbitrary numeric arrays. Since none of these *perturbation* exponents wraps modulo m, their folded ordinary evaluation is exactly

$$\rho^p(K + \rho^{2011s}) = (K + 2^W)\delta.$$

The unmodified raw stencil may cross the cyclic boundary; the argument does not assume otherwise. The needed equality concerns its difference under the update. The exact finite evaluation and the numerical support inequalities are among the checked core lemmas.

Combining this raw increment with equation (1) for old and new words gives

$$F' = F + (K + 2^W)\delta. \tag{2}$$

The current normalization theorem bounds F' strictly below the cyclic modulus. Substitute equation (2) to obtain nonwrapping for this actual unused-dummy mutation. This reverses the dependency: exact update first, size consequence second. It does not immediately recover the more general exported nonwrapping statement, whose hypotheses omit that the dummy is initially zero.

3.4 Concrete integration order

1. Establish the exact folded raw perturbation equality from the literal convolution representation, update-as-addition, and the two support inequalities. Reuse the already-proposed update and finite-sum APIs.
2. Adapt the cyclic carry arrays to the finite conservation contract with $q_i = k((i : \mathrm{ZMod}\, m) - 1)$. Then $q_0 = q_m = k(-1)$; the successor equation is a cast-normalization step.
3. Instantiate the top-field carry theorem at the last cell for both Boolean assemblies, using `last_top_val` to identify the common boundary.
4. Prove `safe_normalized_increment` directly. Do not call the old `normalized_increment` or `normalized_nonwrap_of_safe` while deriving it.
5. Derive a nonwrapping consequence *with* the unused-dummy premise from the updated word's strict bound. Keep the stronger existing no-unused-dummy theorem separate until its compatibility proof is settled.
6. Let the existing existence and rank-decrement endpoints consume the equality; delete quantitative helpers only after checking remaining clients.

All three current mutation, existence, and rank consumers have the unused-dummy premise, so the narrower route suffices for them. The dependency reversal in steps 4–5 is essential. Reusing the old normalized-increment theorem to get the new equality would merely hide the original safety assumption and make the proposal circular.

3.5 Limits and payoff

Equal boundary carries cannot be omitted. For $\rho = 3$ and $m = 1$, an old normalization with raw entry 0, digit 0, and cyclic carry 0 satisfies the recurrence. A new one with raw entry 3, digit 1, and cyclic carry 1 also satisfies it, since $3 + 1 = 1 + 3$. The raw increment is 3 but the normalized increment is 1. The missing 2 is exactly $(3^1 - 1)(1 - 0)$.

The production claim is therefore restricted to the actual Boolean assemblies and physically interior dummy placement, with the existing whole-field temporal phase. It is not a transport theorem for arbitrary normalized cyclic words or arbitrary rotations.

The existing `normalized_nonwrap_of_safe` theorem has no premise dummy$(j) = 0$. If that value is already one, the proposed update changes nothing, so equation (2) is not available with the desired positive increment. Its unchanged statement cannot be called a direct consequence of the actual update argument. Quantitative helper statements also occur in `CurrentDummySafety2011Checks`; removing their definitions entails a deliberate API/check update.

There is a possible independent compatibility proof: normalize a fresh Boolean dummy map identically zero and its update at j to one. Their exact difference is $\Delta = (K + 2^W)\delta$, so $0 \le \Delta \le F_1 < \rho^{m-1}$. Combining this with the original $F < \rho^{m-1}$ and $\rho = 2^{47}$ gives $F + \Delta < \rho^m - 1$. This could preserve the broader statement without the kernel-magnitude chain, but it costs two auxiliary normalizations and a short bound on the sum. It has not been integrated or compiled and is not included in any saving count.

The numerical route includes approximately a hundred source lines, but a net removal cannot be claimed until the cyclic adapter, perturbation proof, and any compatibility wrappers are implemented. Four generic lemmas are checked; the production conversion from `ZMod` arrays and literal convolution is still proposed. The durable benefit, even with previous carry APIs assumed, is elimination of an entire prerequisite family of large-number estimates from the mutation/rank proof.

4 C: one joint envelope for the shifted numerator

4.1 Exact target and unchanged statement

FreeCarrierRatio.shifted_Q_bounds, beginning at source line 275 in both pins, proves positivity and a strict upper bound for a shifted numerator. The included patch keeps every hypothesis and the conclusion unchanged. It adds no helper declaration and no production import.

All variables below are integers. The existing assumptions include

$$A \ge 2,\ C \ge 1,\ A^2 \le D,\ C^2 < D, \quad G \ge A,\ H \ge 1,\ k \ge 3, \quad 0 < W < A.$$

Set

$$J = 4A - 5, \quad Q = 3WC - 2(W^2 - 1), \quad Q_* = J(W^2 + 1) + Q, \quad T = 8D^2 - 4D - 1.$$

The goal is $0 < Q_* < kTGH$.

The original proof successively bounds Q, Q_*, $9D$, and then products involving A, G, H. These estimates can be replaced with a single tighter bound for the complete numerator:

$$Q_* \le (4A - 4)D + 1. \tag{3}$$

The expression is a useful discrete envelope; no global optimality or attainability assertion is intended.

4.2 Derivation

Integrality and $0 < W < A$ imply $W^2 + 1 \le D$. Also

$$0 \le (W - C)^2 = W^2 + C^2 - 2WC$$

with $W^2 \le D - 1$ and $C^2 \le D - 1$ gives $WC + 1 \le D$. Expand the joint numerator once:

$$Q_* = (4A - 7)W^2 + 3WC + 4A - 3.$$

Because $4A - 7 \geq 0$, substituting both endpoints yields

$$\begin{aligned} Q_* &\leq (4A-7)(D-1) + 3(D-1) + 4A - 3 \\ &= (4A-4)D + 1. \end{aligned}$$

The same expanded expression is positive: its quadratic term is nonnegative, $WC > 0$, and $4A - 3 > 0$.

For the denominator, use the explicit polynomial identity

$$3T - 4D = (D-1)(24D+8) + 5.$$

Since $D \geq 1$, this immediately supplies the needed positive gap. More explicitly,

$$3TA - \big((4A-4)D + 1\big) = A(D-1)(24D+8) + 5A + 4D - 1 > 0.$$

Finally $T \geq 0$, $k \geq 3$, $G \geq A > 0$, and $H \geq 1$ give

$$Q_* \leq (4A-4)D + 1 < 3TA \leq kTGH.$$

The Lean proof passes the nonnegative product witnesses explicitly to `nlinarith only`. This keeps the nonlinear argument small: the tactic normalizes known polynomial certificates instead of being expected to discover a factorization or an arbitrary product inequality. Ordinary multiplication monotonicity handles the final independent factors.

4.3 Measured improvement and validation

Unit	Original	Replacement
Complete theorem, physical lines	39	29
Complete `FreeCarrierRatio` module, physical lines	501	491
New helper declarations	0	0
New production imports	0	0

The full replacement theorem with the exact original signature compiled successfully in `ShiftedNumerator.lean`. Its reported axioms are `propext`, `Classical.choice`, and `Quot.sound`. After that check, a documentation adjective changed; the proof code did not. The final package provides the exact proof and its original successful compiler receipt, and records their final hashes.

This is the lowest-integration-cost recommendation: apply the proof-body patch and rebuild the ordinary module. No expression evaluated by the development is changed. The standalone compilation certifies the theorem, but the full production import closure was not rebuilt here. Source-line reduction is measured; elapsed compilation improvement is not.

5 D: optional classical proof for a constant predicate

5.1 Why the stated endpoint admits a much shorter proof

The source defines

```
def refutationInputs : Set Nat := fun _ => C222 F
```

The natural-number input is unused. For any *fixed* proposition P, classical logic proves that the constant predicate $n \mapsto P$ is computable: its characteristic function is the constant Boolean `decide P`. The Mathlib interface permits

```
theorem constant_computablePred (P : Prop) :
    ComputablePred (fun _ : Nat => P) := by
  classical
  exact (Computable.const (decide P)).computablePred
```

and recursive enumerability follows by `.to_re`. In the original source context, the production theorem can be replaced by

```
theorem refutationInputs_re : REPred refutationInputs := by
  classical
  exact (Computable.const (decide (C222 F))).computablePred.to_re
```

There is no reason to add an unused general helper to the production tree for this one application. The generic lemma belongs in the supplied validation artifact to expose exactly what is being used.

The original theorem has 14 physical lines; the replacement has 3. Its docstring currently attributes the proof to the actual verified enumerator. Correcting that provenance adds a line, so the supplied original-source patch removes **10 lines overall**, rather than 11. The appropriate documentation says that the constant input set is classically r.e. and that the executable enumerator is certified separately by `haltingTest_natPrimrec` and `exists_haltingTest_eq_one_iff`.

5.2 The non-effective boundary is substantive

The fixed predicate is either the universal set or the empty set, both computable. Classical logic proves existence of the appropriate constant program without effectively choosing which case holds. This does not decide $C222\ F$, give a computable procedure taking arbitrary propositions as input, or construct an explicit refutation enumerator.

In the original development, `haltingTest_natPrimrec` still needs its computational certificate. D alone therefore cannot justify deleting the private primitive-recursion chain. Proposal A supplies a separate and genuinely data-backed justification for deleting that chain. The two ideas simplify different obligations, and should not be conflated.

Adopt D only if the r.e. theorem's proposition is the intended contract. If this particular theorem is also intended to document the derivation through the actual refutation search, A's retained search proof is the better choice. A second unused classical theorem would increase the development instead of simplifying it. The archive labels the original-source D patch (03a) as an alternative to overlapping edits in A. It also includes a separate patch (03b) to apply only after A, using `decide (refutationInputs 0)` so no moved namespace openings are required. That version removes nine lines from the candidate file, including its corrected documentation. A small stand-alone template checks the public-predicate formulation. These alternatives are not part of an additive savings total.

5.3 Compatibility with the choice-defined polynomial branch

Polynomial.lean defines its Jones-derived representations by `Classical.choose` and explicitly distinguishes them from executable coefficient recipes. Replacing only the proof of the *same* recursive-enumerability proposition does not change a proof-indexed selector. Lean proof irrelevance gives

```
theorem selector_proof_irrelevant {a : Sort u} {S : Prop}
    (f : S -> a) (h1 h2 : S) : f h1 = f h2 := rfl
```

The same definitional invariance holds for two proofs of an identical existential proposition passed to `Classical.choose`. The supplied core checks both statements and a representation-selector version with the shape of the Jones construction.

This compatibility statement has a precise boundary: the proposition and specification types must be identical. It does not cover replacing explicit program data, changing the Jones specification, or redesigning a polynomial based on a proposition-dependent constant. It supplies no concrete coefficient recipe. The selector theorem itself has no axioms; the classical computability proof uses the standard axioms reported in its log.

6 Evidence, reproduction, and adoption priorities

6.1 What was checked

The compiler was Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. Mathlib was pinned at `81a5d257c8e410db227a6665ed08f64fea08e997`. The nine package Git pins were compared with the repository manifest. The checks used the matching existing dependency cache and direct Lean invocations; they were not a fresh `lake build` of the entire repository. The actual `ScalarPRCode` source was compiled locally before checking A's core.

Artifact	Successful check and remaining boundary
`lean/CertificateReuse.lean`	Actual scalar-realizer bridge, eventual-success search, and composition. Full migration remains unbuilt.
`lean/BoundaryConservation.lean`	Four conservation/support/evaluation lemmas. Actual cyclic-array adapter and complete safety replacement remain proposed.
`lean/ShiftedNumerator.lean`	Exact theorem signature and full replacement proof. Complete production module remains unbuilt.
`lean/ConstantPredicate.lean`	Constant computability/r.e. and proof-irrelevance compatibility. Production patch remains unbuilt.

Each core completed with exit code zero. The files contain no `sorry`, `admit`, new `axiom` declaration, or `native_decide` shortcut. The logs include `#print axioms` output. Successful theorem checking is stronger than testing a handful of numerical examples, but its scope is exactly the statements in those files.

For A, extra static checks establish patch applicability, an acyclic local import graph, and preservation of the actual scalar-program body after imports. For C and D, patch applicability and exact physical line differences are separate checks. Eight target/boundary Git blobs are unchanged between the initial and final pins; only five other maintained Lean files changed during the refresh. This is source-identity evidence, not a new dependency-closure build.

6.2 Reproduce and integrate

The archive's `README.txt` gives patch alternatives and commands. `scripts/check_lean.py` is a portable reproduction driver: point it at a checkout with the pinned toolchain and dependencies. It first builds the scalar-code prerequisite, then invokes `lake env lean` on the four principal cores and the small post-migration template. This driver is supplied for reproduction; the recorded local receipts document the direct invocations actually used in this review.

The package includes a nonmutating source/patch checker, complete migration candidates, pinned environment files, audit manifests, and a SHA-256 inventory.

Recommended order:

1. Apply C first: a small complete theorem replacement with a measured ten-line reduction and no helper or import overhead.
2. Integrate A: the largest measured candidate source reduction, with a clear module split and an explicit full-build gate.
3. Develop B against the assumed prior carry/update APIs. Its value is removal of the numerical dependency, so count all adapter and wrapper costs before reporting net savings.
4. Decide whether D fits the intended provenance of the existence-only branch. If it does, use the expressly sequential 03b patch after A rather than applying overlapping original-source diffs blindly.

No new simplification is attributed to profiling, import hygiene, theorem naming, generic automation, or compatibility checks. Those are validation and implementation work supporting the four proposals. No modifications have been pushed to the repository.

6.3 Open implementation questions

Three narrow questions remain worth resolving during integration. Can the existing exact cyclic telescope be specialized with only a short indexing adapter, or does its public contract expose only modular equality? Which quantitative safety helpers still have independent clients after the ordinary mutation equality becomes primary? Do the project's current executable `rfl` checks rely on any private instance identity that the conservative extraction needs to keep local? These questions determine engineering cost; they are not additional novelty claims.

Source map and archive contents

All source links in this report point to the final comparison pin. The principal targets are Computability, DirectScalarEnumerator, ScalarPRCode, CurrentDummySafety2011, CurrentDummyMutation2011, CurrentWordTopBounds2011, CurrentKernel2011, FreeCarrierRatio, and Polynomial. The full comparison tree is research/lean-proof-simplification; the other requested exclusion location is docs/incoming.

Directory/file	Contents
`report/`	This PDF and complete standalone LaTeX source; build instructions
`lean/`	Four principal checked Lean cores plus a small post-migration template
`patches/`	Candidate certificate migration, shifted-numerator replacement, optional original-source and post-A constant-predicate alternatives
`validation/`	Successful logs/receipts, exact counts, patch and static checks, environment identity
`audit/`	Exclusion taxonomy, refreshed decisions, source/placement identities, prior-text inventory and heading index
`reference/`	Complete candidate migration files, original numerator theorem, pinned toolchain and dependency manifest
`scripts/`	Reproduction and nonmutating patch-check drivers
`README.txt`	Status, patch alternatives, commands, and scope
`MANIFEST.sha256`	SHA-256 checksums for the deliverable files